

## ENGINEERING MANUAL

# Unitree L1 Engineering Manual

ROS 2 Humble, RViz2, diagnostics, recording, and OctoMap

---

First edition

Prepared 29 July 2026

Validated project baseline 07616c2

Audience: Robotics engineers, integrators, operators, and maintainers

## DOCUMENT CONTROL

This document is generated from `engineering-manual.md`.  
Commands and paths describe the repository state identified above.

**Operate the powered LiDAR only after reading the safety section.**

# 1. About this manual

This manual is the engineering reference for the Unitree 4D LiDAR L1 ROS 2 project at commit 07616c2. It explains the system boundary, the hardware and software architecture, the behavior of each project package, normal operating procedures, validation, fault isolation, and safe maintenance.

The project runs the complete ROS 2 Humble stack in Docker. The Ubuntu 24.04 host remains the Docker client, X11 display server, storage system, and owner of the physical serial device. ROS 2, the Unitree driver, diagnostics, RViz2, rosbag2, TF, and OctoMap run in an Ubuntu 22.04 container.

The manual is written for engineers who need to understand not only which command to run, but why the command exists, what crosses each boundary, what evidence constitutes success, and what the software intentionally refuses to do.

## 1.1 Document authority

The executable files and configuration in the repository are authoritative. This manual describes those files at the project commit shown on the cover. When behavior and prose disagree, stop the operation, inspect the current source, and update the documentation and validation record together.

The principal supporting records are:

- `README.md` for the concise entry point.
- `docs/decisions.md` for accepted architecture decisions.
- `docs/hardware-runbook.md` for the bench procedure.
- `docs/project-structure.md` for ownership and generated-data rules.
- `docs/sources.md` for upstream technical references.
- `docs/validation-matrix.md` for reproducible acceptance evidence.
- `docs/versions-lock.md` for immutable inputs and compatibility.

## 1.2 Purpose

The system provides a reproducible way to:

- Detect and safely expose one Unitree serial adapter to Docker.
- Parse Unitree L1 serial data with the pinned UniLiDAR SDK.
- Publish a ROS 2 point cloud and IMU stream.
- Assess stream presence, rate, timing, frame, and point metadata.
- Display the raw point cloud in RViz2.
- Record bounded or operator-controlled rosbag2 datasets.
- Replay recorded data without the physical LiDAR.
- Insert a time-aligned point cloud into a probabilistic OctoMap.
- Display occupied OctoMap voxels in RViz2.
- Save, inspect, and reopen OctoMap files.
- Rebuild and validate the complete environment from pinned source.

## 1.3 Non-goals

The current project does not provide odometry, localization, trajectory estimation, loop closure, navigation, obstacle avoidance, or motion planning. It is not a complete SLAM system.

OctoMap creates and publishes a three-dimensional occupancy map. It does not estimate the pose of the LiDAR. A moving sensor requires a separate source of time-varying pose, exposed through TF at the timestamp of every cloud.

The project also does not:

- Install ROS 2 Humble on the Ubuntu 24.04 host.
- Configure a persistent host udev rule.
- Disable ModemManager automatically.
- Change host permissions with `chmod 777`.
- Run Docker with unrestricted privileged access.
- Configure the Unitree LEDs or manage firmware.
- Correct the vendor driver's initialization behavior silently.
- Record arbitrary odometry or application topics in the standard bag wrapper.
- Judge map accuracy from the presence of occupied voxels.
- Preserve generated bags, maps, reports, logs, or build products in Git.

### ENGINEERING NOTE

Point-LIO integration remains pending. A candidate must be selected and tuned from validated L1 messages, timestamps, fields, units, rates, and LiDAR-to-IMU extrinsics rather than from assumed compatibility.

## 1.4 Meaning of validation terms

PASS in the validation matrix records a reproducible software check. PASS\_HW records a check that was completed with the physical L1. A process being alive is not sufficient evidence. Live validation requires actual messages, non-zero measured rates, and diagnostics.

The word healthy in this manual means that the implemented checks passed. It does not certify functional safety, metrology, trajectory accuracy, map accuracy, or suitability for an autonomous vehicle.

## 2. Safety and operating limits

### 2.1 Mechanical and electrical safety

#### SAFETY / WARNING

Secure the L1 before applying power. Its mechanism can begin moving. Keep hands, loose clothing, cables, tools, and nearby objects outside the entire operating area.

Disconnect power before changing any wiring. Use the Unitree cable and adapter described by the manufacturer. The expected connection is:

```
Unitree L1 -> Unitree serial adapter -> PC USB port
Unitree L1 -> separate, correctly polarized 12 V / 1 A supply
```

#### SAFETY / WARNING

Do not power the L1 from 5 V USB. Do not connect the L1's 3.3 V TTL interface directly to USB or RS-232. Use the supplied or correctly specified adapter.

The current serial driver uses 2,000,000 baud. Do not alter the rate without hardware documentation and a controlled validation plan.

### 2.2 Host and container safety

The project deliberately uses least-privilege controls:

- No privileged: true.
- All Linux capabilities are dropped.
- no-new-privileges is enabled.
- The GUI overlay exposes only X11 authorization and DRI devices.
- The hardware overlay exposes only the selected tty device.
- The tty group is added by numeric GID rather than broad permissions.
- The X11 cookie and X11 socket are mounted read-only.
- Host networking is not enabled by default.

#### SAFETY / WARNING

Do not use a global `xhost +`. Do not use `chmod 777` on the serial device. Do not disable a service or kill a process merely because it might own the port.

The detection script reports evidence and changes no host permissions, services, udev rules, or network settings. If a specific process repeatedly opens the identified adapter, record its USB VID, PID, serial number, and port ownership before considering a narrowly targeted correction.

### 2.3 Mapping safety boundary

Stationary mapping uses a project-generated identity transform from `map` to `unilidar_lidar`. That transform is a bench convenience, not localization.

#### SAFETY / WARNING

Never move the LiDAR, its mount, or the robot while `STATIC_SENSOR=true`. Motion under an identity transform assigns every scan to the same pose and creates a geometrically false map.

For motion, set `STATIC_SENSOR=false` and provide a time-aligned dynamic TF from an external pose estimator. The IMU topic alone is not a pose transform.

### 2.4 Data and process safety

Stop foreground launch, recording, and viewing processes with `Ctrl-C` so ROS nodes and rosbag2 can shut down cleanly. A bounded recording receives `SIGINT` so `metadata.yaml` can be finalized.

The save wrapper refuses to overwrite an existing map. Map and bag names are validated and confined to project-owned ignored directories. Do not bypass those path controls in routine operation.

## 3. System architecture

### 3.1 Physical-to-ROS data flow

The complete live path is:

```
Physical Unitree L1
|
| Unitree adapter, separate power, 2,000,000 baud
v
Host /dev/serial/by-id/... -> resolved ttyUSB* or ttyACM*
|
| Docker device mapping and supplementary tty GID
v
Container /dev/unitree_lidar
|
v
/unitree_lidar_ros2_node
|
+--> /unilidar/cloud
|   +--> /l1_monitor
|   +--> raw RViz2
|   +--> rosbag2
|   +--> cloud_in on /octomap_server
|
+--> /unilidar/imu
|   +--> /l1_monitor
|   +--> rosbag2
```

The diagnostic and mapping branches are:

```
/l1_monitor
|
+--> /diagnostics

/unilidar/cloud + map-to-lidar TF
|
v
/octomap_server
|
+--> /occupied_cells_vis_array -> map RViz2
+--> /octomap_binary
+--> /octomap_full
+--> /octomap_point_cloud_centers
+--> /projected_map
+--> /octomap_binary service -> saver -> maps/*.bt or maps/*.ot
```

The physical data enters through one serial character device. The repository, bags, maps, and logs cross the container boundary through the `/workspace` bind mount. The ROS graph itself remains inside the container.

### 3.2 Runtime processes

The normal live driver launch creates:

| ROS node                 | Condition     | Responsibility                      |
|--------------------------|---------------|-------------------------------------|
| /unitree_lidar_ros2_node | Always        | Serial parse and sensor publication |
| /l1_monitor              | monitor:=true | Read-only stream diagnostics        |
| /rviz2                   | rviz:=true    | Raw point-cloud visualization       |

The separate live mapping launch adds:

| ROS node                   | Condition           | Responsibility                      |
|----------------------------|---------------------|-------------------------------------|
| /octomap_server            | Always              | Occupancy insertion and publication |
| /l1_static_lidar_transform | static_sensor:=true | Bench identity TF                   |
| /octomap_rviz2             | rviz:=true          | Live cloud and map visualization    |

The saved-map workflow uses a separate Compose container and ROS domain. It creates `/octomap_server` and, by default, `/saved_octomap_rviz2`.

### 3.3 Frame model

The driver places the point cloud in `unilidar_lidar` and the IMU in `unilidar_imu`. It publishes no transform between those frames.

The raw RViz profile uses `unilidar_lidar` as its fixed frame, so no TF is needed to inspect the cloud in its native sensor coordinates.

OctoMap uses `map` as the world frame and must resolve the cloud frame into that world frame at the cloud timestamp. In stationary mode, the project publishes this identity transform:

```
parent: map
child:  unilidar_lidar
x y z:  0 0 0
roll pitch yaw: 0 0 0
```

In mobile mode, an external TF tree may contain intermediate frames, but the complete chain from `map` to `unilidar_lidar` must be available and correctly timestamped.

### 3.4 Topic and QoS model

The Unitree driver creates depth-10 publishers. In ROS 2 Humble this produces Reliable, Volatile, Keep Last behavior for the cloud and IMU.

The monitor intentionally matches that contract with Reliable, Volatile, KeepLast(10) subscriptions. Its diagnostic publisher uses depth 10.

The OctoMap server subscribes to `cloud_in` with sensor-data QoS, which is Best Effort. A Best Effort subscriber is compatible with the driver's Reliable publisher. The live map RViz profile also uses Best Effort for the cloud.

With `latch: true`, OctoMap map outputs use Transient Local durability. A late RViz or evaluation subscriber can therefore receive the latest map without waiting for a new insertion.

## 4. Docker environment and security design

### 4.1 Base image

`docker/Dockerfile` begins from:

```
ros:humble-ros-base-jammy
sha256:5c793b92e0b12d6babb438cb20eed7766495fde6419a21e3d2e918464f09dc17
```

The validated environment is Ubuntu 22.04 Jammy on amd64 with ROS 2 Humble. The image installs the compiler toolchain, `colcon`, `rosdep`, `vcstool`, PCL, OctoMap, RViz2, `rosbag2`, TF, X11, DRI support, USB inspection utilities, and the ROS interfaces required by the packages.

The base image and Git commits are immutable inputs. The individual packages installed by `apt-get` are not pinned to a repository snapshot. A future image build is source-reproducible but is not guaranteed to be byte-for-byte identical.

### 4.2 User and file ownership

The build accepts `HOST_UID` and `HOST_GID`. It creates the unprivileged user `ros` with those identifiers and gives that user ownership of `/workspace`. The wrappers export the current host identifiers automatically.

This arrangement prevents ordinary build output, maps, bags, and logs from becoming root-owned on the host.

### 4.3 Base Compose service

`docker/compose.yaml` defines service `dev` in project `unitree-l1`. Important properties are:

- Image name `unitree-l1:humble-v1.0.16`.
- Working directory `/workspace/ros2_ws`.
- Repository bind mount from the project root to `/workspace`.
- `COLCON_DEFAULTS_FILE=/workspace/ros2_ws/colcon_defaults.yaml`.
- `ROS_DOMAIN_ID=42` unless the operator overrides it.
- Docker init enabled for child-process handling.
- All capabilities dropped.
- No new privileges.
- Default Docker bridge networking.

All nodes that need to communicate in the standard workflow run in the same named runtime container. Host networking is unnecessary for the serial L1.

### 4.4 GUI overlay

`docker/compose.gui.yaml` adds:

- Host `DISPLAY`.
- Container `XAUTHORITY=/tmp/host.xauthority`.
- Read-only `/tmp/.X11-unix`.
- Read-only host X11 cookie.
- `/dev/dri` device access.
- Numeric video and render supplementary groups.

The wrappers derive `VIDEO_GID` from `/dev/dri/card0` and `RENDER_GID` from `/dev/dri/renderD128`.

The GUI assertion requires:

```
DISPLAY is set
XAUTHORITY is set and readable
/dev/dri/renderD128 exists in the container
rviz2 resolves below /opt/ros/humble
```

Run the independent check with:

```
./scripts/gui-smoke-test.sh
```

It validates X11 with `xdpyinfo`, reports OpenGL with `glxinfo -B`, and checks that RViz2 starts far enough to provide its help interface.

## 4.5 LiDAR overlay

`docker/compose.lidar.yaml` maps the selected host device to:

```
/dev/unitree_lidar
```

It adds only the source tty's numeric group and sets:

```
LIDAR_PORT=/dev/unitree_lidar
```

The runtime wrapper validates that the selected target:

- Resolves to a real path.
- Is a character device.
- Has a basename matching `ttyUSB*` or `ttyACM*`.
- Is not currently reported as open by `fuser`.
- Is readable and writable inside an unprivileged test container.

The preflight verdict is:

```
LIDAR_CONTAINER_ACCESS_PASS
```

## 4.6 Docker-only enforcement

`scripts/assert-ros-container.sh` refuses execution unless:

- `/.dockerenv` exists.
- `/etc/os-release` reports Ubuntu 22.04.
- `/opt/ros/humble/setup.bash` exists.
- `ROS_DISTRO` is `humble`.
- `ros2` and `rviz2` resolve below `/opt/ros/humble`.
- Requested GUI resources are present.

Project launch files also inspect `/.dockerenv`. A direct attempt to launch them on the host raises a Docker-only error.

### EXPECTED RESULT

A successful assertion prints `DOCKER_ROS_RUNTIME_PASS`. A host invocation fails deliberately with `DOCKER_RUNTIME_ASSERT_FAIL`.

## 4.7 Build context

`.dockerignore` starts by excluding everything, then permits only the Docker directory, Dockerfile, and entrypoint. Source trees, bags, maps, logs, local build output, credentials, and other workspace data are not uploaded as image build context.

# 5. Source dependencies and reproducibility

## 5.1 Immutable dependency manifest

`config/dependencies.repos` identifies the external repositories:

| Component            | Version | Commit                                   |
|----------------------|---------|------------------------------------------|
| Unitree UniLiDAR SDK | v1.0.16 | 1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6 |
| OctoMap mapping      | 2.3.1   | f79da9a9a1fcdf82e72dab4df288d6cc27c6e163 |

The checkouts are recreated under:

```
ros2_ws/src/unilidar_sdk/
ros2_ws/src/octomap_mapping/
```

They are ignored by the project Git repository. Their origin and exact commit, not copied generated content, are the reproducibility mechanism.

## 5.2 Dependency import behavior

Run:

```
./scripts/fetch-dependencies.sh
```

The wrapper starts a headless container and uses `vcs import` only when a checkout is missing. It then verifies each commit and requires a clean vendor worktree.

It fails if:

- A target exists but is not a Git checkout.
- The checkout is at a different commit.
- The checkout contains local changes.
- Import or verification fails.

### SAFETY / WARNING

A failed verification is a change-control event. Do not edit the vendor tree merely to make a build pass. Determine whether the local checkout should be preserved, deliberately updated, or recreated from the manifest.

## 5.3 Package discovery

The Unitree repository contains a ROS 1 package and a raw CMake SDK in addition to the ROS 2 driver. The OctoMap repository contains a metapackage and server.

`ros2_ws/colcon_defaults.yaml` lists only these roots:

```
/workspace/ros2_ws/src/l1_bringup
/workspace/ros2_ws/src/l1_monitor
/workspace/ros2_ws/src/l1_octomap_bringup
/workspace/ros2_ws/src/octomap_mapping
/workspace/ros2_ws/src/unilidar_sdk/unitree_lidar_ros2
```

The expected six-package result is:

```
l1_bringup
l1_monitor
l1_octomap_bringup
octomap_mapping
octomap_server
unitree_lidar_ros2
```

## 5.4 Build output isolation

All colcon paths are absolute inside the mandatory project container:

```
/workspace/ros2_ws/build
/workspace/ros2_ws/install
/workspace/ros2_ws/log
```

The paths remain correct even if an operator invokes colcon while standing in `/workspace/ros2_ws/src`.

### SAFETY / WARNING

A directory named `build`, `install`, or `log` anywhere below `ros2_ws/src` violates the project structure.

Three controls enforce the invariant:

1. Compose exports the colcon defaults file.
2. The defaults file supplies absolute discovery and output paths.
3. `workspace-build.sh` scans the source tree before and after compilation.

The build uses symlink install for efficient project-source iteration.

## 5.5 Canonical build

From the repository root:

```
./scripts/docker-build.sh
./scripts/workspace-build.sh
./scripts/smoke-test.sh
```

The workspace wrapper:

1. Refuses forbidden generated directories below `src`.
2. Fetches and verifies both external dependencies.
3. Runs `rosdep` for the five source roots.
4. Skips only the known `ament_python` and `pcl` `rosdep` keys.
5. Builds all six packages with colcon.
6. Rechecks source-tree isolation.

If a future dependency needs a new system library, add it to `docker/Dockerfile`. Do not rely on a package installed only in a disposable Compose run container.

## 5.6 Interactive build and tests

Open a headless, hardware-free shell:

```
./scripts/docker-shell.sh --no-gui --no-lidar
```

Inside:

```
cd /workspace/ros2_ws
colcon list
colcon build
source install/setup.bash
colcon test \
  --packages-select ll_bringup ll_monitor ll_octomap_bringup
colcon test-result --all --verbose
```

The current project result is 17 tests with zero failures. The three project packages contribute launch-contract tests, metadata tests, monitor statistics tests, and Python style checks.

## 6. ROS package engineering

### 6.1 Package ownership model

Project adaptations are separated from fetched upstream source:

| Package            | Build type   | Project responsibility              |
|--------------------|--------------|-------------------------------------|
| ll_bringup         | ament_python | Driver launch, parameters, raw RViz |
| ll_monitor         | ament_python | Stream statistics and diagnostics   |
| ll_octomap_bringup | ament_python | Mapping launch, parameters, RViz    |
| unitree_lidar_ros2 | ament_cmake  | Upstream serial driver              |
| octomap_server     | ament_cmake  | Upstream occupancy implementation   |
| octomap_mapping    | ament_cmake  | Upstream mapping metapackage        |

The project does not modify either external checkout. Adaptations belong in a project-owned `ll_*` package or a host-facing script.

### 6.2 ll\_bringup

The main files are:

```
ros2_ws/src/ll_bringup/launch/unitree_ll.launch.py
ros2_ws/src/ll_bringup/config/unitree_ll.yaml
ros2_ws/src/ll_bringup/config/unitree_ll.rviz
```

The launch file enforces Docker, starts the Unitree executable, passes the project configuration, and conditionally starts the monitor and RViz2.

Its public arguments are:

| Argument    | Default                      | Function          |
|-------------|------------------------------|-------------------|
| port        | \$LIDAR_PORT or /dev/ttyUSB0 | Serial device     |
| cloud_topic | unilidar/cloud               | PointCloud2 name  |
| imu_topic   | unilidar/imu                 | IMU name          |
| cloud_frame | unilidar_lidar               | Cloud frame       |
| imu_frame   | unilidar_imu                 | IMU frame         |
| rviz        | false                        | Start raw RViz    |
| monitor     | true                         | Start diagnostics |

ROS resolves the relative defaults `unilidar/cloud` and `unilidar/imu` at the root namespace in the standard launch, producing `/unilidar/cloud` and `/unilidar/imu`.



## 6.3 Unitree driver parameters

l1\_bringup/config/unitree\_l1.yaml supplies:

| Parameter       | Value              | Engineering interpretation    |
|-----------------|--------------------|-------------------------------|
| port            | /dev/unitree_lidar | Stable container device       |
| rotate_yaw_bias | 0.0                | SDK yaw correction in degrees |
| range_scale     | 0.001              | Raw range multiplier          |
| range_bias      | 0.0                | SDK range offset              |
| range_max       | 50.0               | Driver maximum range          |
| range_min       | 0.0                | Driver minimum range          |
| cloud_frame     | unilidar_lidar     | Point-cloud frame             |
| cloud_topic     | unilidar/cloud     | Point-cloud topic             |
| cloud_scan_num  | 18                 | SDK cloud aggregation count   |
| imu_frame       | unilidar_imu       | IMU frame                     |
| imu_topic       | unilidar/imu       | IMU topic                     |

The project launch overrides port, topics, and frames from its public arguments. The remaining values come from the YAML file.

The SDK source adds `rotate_yaw_bias` to the measured horizontal angle before converting degrees to radians. The scale, bias, and range limits are passed to the SDK initialization call. Treat them as calibrated sensor parameters; do not tune them to disguise a frame, wiring, timestamp, or pose problem.

## 6.4 Pinned Unitree ROS 2 driver

The upstream implementation is primarily in:

```
ros2_ws/src/unilidar_sdk/unitree_lidar_ros2/src/unitree_lidar_ros2/
```

The node:

- Creates a `UnitreeLidarReader`.
- Calls `initialize` with 2,000,000 baud.
- Polls `runParse()` from a 1 ms wall timer.
- Converts parsed Unitree points to a PCL point type.
- Converts PCL output to `sensor_msgs/msg/PointCloud2`.
- Converts parsed inertial values to `sensor_msgs/msg/Imu`.
- Copies SDK timestamps to ROS message headers.
- Publishes both streams with depth 10.

The `PointCloud2` fields originate from the registered PCL point type:

| Field     | Source type     | Meaning                            |
|-----------|-----------------|------------------------------------|
| x         | float           | Cartesian x                        |
| y         | float           | Cartesian y                        |
| z         | float           | Cartesian z                        |
| intensity | float           | Return intensity                   |
| ring      | unsigned 16-bit | Scan ring index                    |
| time      | float           | Point time relative to cloud stamp |

The IMU message receives:

- Quaternion in x, y, z, w order.
- Three-axis angular velocity.
- Three-axis linear acceleration.
- SDK timestamp and configured frame.

The current driver does not explicitly populate the covariance arrays. It also publishes no TF between the LiDAR, IMU, robot base, or world.

## 6.5 Driver initialization limitation

The SDK's `initialize()` contract returns zero for success and minus one for a serial-open failure. The current ROS 2 driver calls it without checking the return value.

The upstream SDK example explicitly changes the device from standby to normal mode. The pinned ROS 2 driver does not explicitly request `NORMAL`.

### SAFETY / WARNING

A running `/unitree_lidar_ros2_node`, advertised publishers, or a successfully opened container is not proof that sensor data is flowing.

The acceptance condition is actual messages and measured rates:

```
./scripts/lidar-validate.sh
```

If a mode or initialization correction becomes necessary, implement it in project-owned code, check every return value, document the change, and repeat hardware validation. Do not patch the ignored pinned checkout silently.

## 6.6 Raw RViz profile

l1\_bringup/config/unitree\_l1.rviz is designed for sensor-native inspection:

| Property    | Value                                   |
|-------------|-----------------------------------------|
| Fixed frame | unilidar_lidar                          |
| Cloud topic | /unilidar/cloud                         |
| Cloud QoS   | Reliable, Volatile, Keep Last 10        |
| Point style | Points, 3 pixels                        |
| Point decay | 1 second                                |
| Color       | Axis color from intensity-capable cloud |
| View        | Orbit around unilidar_lidar             |

The display also contains a grid and axes. It does not visualize the IMU and does not require a TF publisher because the cloud and fixed frame are equal.

## 6.7 l1\_monitor

The monitor implementation is:

```
ros2_ws/src/l1_monitor/l1_monitor/monitor_node.py
ros2_ws/src/l1_monitor/l1_monitor/stats.py
```

It observes the raw streams without modifying, filtering, synchronizing, or republishing them.

Its parameters are:

| Parameter         | Default         | Function                     |
|-------------------|-----------------|------------------------------|
| cloud_topic       | /unilidar/cloud | Cloud input                  |
| imu_topic         | /unilidar/imu   | IMU input                    |
| diagnostics_topic | /diagnostics    | Status output                |
| report_period_sec | 2.0             | Report interval              |
| timeout_sec       | 3.0             | No-data or stale threshold   |
| max_stamp_age_sec | 1.0             | Absolute header-age warning  |
| min_cloud_hz      | 5.0             | Cloud-rate warning threshold |
| min_imu_hz        | 20.0            | IMU-rate warning threshold   |
| window_size       | 100             | Bounded arrival window       |

report\_period\_sec and timeout\_sec must be positive. window\_size must be at least two.

## 6.8 Monitor statistics

Arrival timing uses the host's monotonic clock inside the container. Header age uses the ROS clock. This separation prevents wall-clock changes from corrupting the measured arrival interval while still exposing header-clock problems.

For an arrival window containing times  $a_0$  through  $a_n$ , the reported frequency is:

$$\text{frequency} = n / (a_n - a_0)$$

The implementation uses nanoseconds internally and reports hertz. Before two arrivals, the frequency is zero.

The cloud callback records:

- Cumulative received count.
- Arrival timestamps.
- Latest header timestamp.
- Latest frame.
- Width multiplied by height as point count.
- Ordered PointCloud2 field names.

The IMU callback records the same timing and frame data without point metadata.

## 6.9 Diagnostic levels

Two DiagnosticStatus records are published:

```
unitree_l1/cloud
unitree_l1/imu
```

Both use hardware ID unitree\_l1.

The level logic is:

| Condition                          | Level |
|------------------------------------|-------|
| No messages before timeout         | WARN  |
| No messages after timeout          | ERROR |
| Latest arrival older than timeout  | ERROR |
| Frequency below configured minimum | WARN  |
| One or more non-increasing stamps  | ERROR |
| One or more zero stamps            | WARN  |
| Absolute header age above limit    | WARN  |
| No active problem                  | OK    |

If several conditions occur, the highest severity wins. The message text combines all detected problems.

Each status includes:

```
received
frequency_hz
arrival_age_sec
header_age_sec
frame_id
non_monotonic_stamps
zero_stamps
```

Cloud status additionally includes:

```
point_count
fields
```

The thresholds are conservative alarms, not manufacturer specifications. Validated real rates are approximately 8 to 10 Hz for clouds and 210 to 250 Hz for IMU. The official initial references are approximately 11 Hz and 250 Hz.

## 6.10 ll\_octomap\_bringup

The package owns the Unitree-to-OctoMap integration without modifying upstream OctoMap:

```
ros2_ws/src/ll_octomap_bringup/launch/ll_octomap.launch.py
ros2_ws/src/ll_octomap_bringup/launch/unitree_ll_octomap.launch.py
ros2_ws/src/ll_octomap_bringup/launch/view_saved_octomap.launch.py
ros2_ws/src/ll_octomap_bringup/config/octomap.yaml
ros2_ws/src/ll_octomap_bringup/config/ll_octomap.rviz
ros2_ws/src/ll_octomap_bringup/config/saved_octomap.rviz
```

`ll_octomap.launch.py` attaches OctoMap to an existing PointCloud2 stream. `unitree_ll_octomap.launch.py` combines driver, monitor, mapping, and one RViz process. `view_saved_octomap.launch.py` loads a saved map without hardware.

The package depends on `ll_bringup`, but intentionally has no direct package dependency on `unitree_lidar_ros2`. It accepts any compatible PointCloud2 source and required TF.

## 6.11 Live OctoMap launch arguments

| Argument                   | Default                      | Function                  |
|----------------------------|------------------------------|---------------------------|
| <code>cloud_topic</code>   | <code>/unilidar/cloud</code> | PointCloud2 input         |
| <code>world_frame</code>   | <code>map</code>             | Accumulation frame        |
| <code>lidar_frame</code>   | <code>unilidar_lidar</code>  | Expected sensor frame     |
| <code>resolution</code>    | <code>0.10</code>            | Voxel size in metres      |
| <code>max_range</code>     | <code>15.0</code>            | Maximum ray length        |
| <code>static_sensor</code> | <code>true</code>            | Publish bench identity TF |
| <code>rviz</code>          | <code>false</code>           | Start map RViz            |

The launch remaps the upstream relative subscription `cloud_in` to the chosen cloud topic. It passes the selected world frame, LiDAR frame, resolution, and maximum range over the YAML defaults.

## 6.12 OctoMap configuration

ll\_octomap\_bringup/config/octomap.yaml sets:

| Parameter                 | Value          | Effect                     |
|---------------------------|----------------|----------------------------|
| frame_id                  | map            | World output frame         |
| base_frame_id             | unilidar_lidar | Ground-filter base         |
| resolution                | 0.10           | 10 cm leaf voxel           |
| sensor_model.max_range    | 15.0           | Integration range          |
| sensor_model.hit          | 0.7            | Occupied evidence          |
| sensor_model.miss         | 0.4            | Free evidence              |
| sensor_model.min          | 0.12           | Lower probability clamp    |
| sensor_model.max          | 0.97           | Upper probability clamp    |
| filter_ground_plane       | false          | Do not segment ground      |
| filter_speckles           | false          | Keep isolated cells        |
| compress_map              | true           | Lossless tree pruning      |
| incremental_2D_projection | false          | Full 2D projection         |
| publish_free_space        | false          | No free-cell markers       |
| latch                     | true           | Transient-local map output |

With ground filtering disabled, ground returns are handled as ordinary endpoints and may become occupied. `base_frame_id` becomes important if a future configuration enables ground-plane filtering.

## 6.13 OctoMap outputs

The upstream server publishes:

| Topic                        | Type          | Purpose                    |
|------------------------------|---------------|----------------------------|
| /occupied_cells_vis_array    | MarkerArray   | Occupied voxel display     |
| /free_cells_vis_array        | MarkerArray   | Free display if enabled    |
| /octomap_binary              | Octomap       | Compact occupancy map      |
| /octomap_full                | Octomap       | Full serialized map        |
| /octomap_point_cloud_centers | PointCloud2   | Occupied centers           |
| /projected_map               | OccupancyGrid | Two-dimensional projection |

It provides:

| Service                    | Purpose              |
|----------------------------|----------------------|
| /octomap_binary            | Retrieve compact map |
| /octomap_full              | Retrieve full map    |
| /octomap_server/clear_bbox | Clear a bounding box |
| /octomap_server/reset      | Reset map state      |

ROS permits a topic and service to share the name `/octomap_binary` because they occupy separate name spaces in the graph API.

## 6.14 OctoMap RViz profiles

The live map profile uses:

| Property         | Value                     |
|------------------|---------------------------|
| Fixed frame      | map                       |
| Cloud            | /unilidar/cloud           |
| Cloud QoS        | Best Effort, Volatile     |
| Occupied markers | /occupied_cells_vis_array |
| Marker QoS       | Reliable, Transient Local |
| Camera           | Orbit around map          |

The saved-map profile omits the live cloud and displays only the occupied marker array in `map`.

# 7. OctoMap engineering model

## 7.1 What OctoMap represents

OctoMap divides three-dimensional space into cubic cells arranged in an octree. A leaf at the configured maximum depth represents a cube whose side is the configured resolution. This project uses 0.10 m, so the finest normal cell is a 10 cm cube.

An octree stores large uniform regions more compactly than a dense three-dimensional array. Eight sibling cells with equivalent state can be represented by their parent. `compress_map: true` enables lossless pruning of such equivalent branches.

The map stores occupancy belief, not a surface mesh. A cell is updated from evidence that a ray passed through it or ended in it.

## 7.2 Bayesian log-odds update

For occupancy probability  $p$ , the log-odds value is:

$$L(p) = \ln(p / (1 - p))$$

With a neutral prior of 0.5, the prior log odds are zero. A hit adds:

$$L(0.7) = \ln(0.7 / 0.3) = \text{approximately } 0.8473$$

A miss adds:

$$L(0.4) = \ln(0.4 / 0.6) = \text{approximately } -0.4055$$

The conceptual update for voxel  $n$  at step  $t$  is:

```
L_t(n) = clamp(
    L_(t-1)(n) + L(sensor_evidence) - L(prior),
    L(min),
    L(max)
)
```

For the current neutral prior, subtracting the prior changes nothing. The configured clamps are:

$$L(0.12) = \text{approximately } -1.9924$$

$$L(0.97) = \text{approximately } 3.4761$$

Repeated hits increase confidence toward 0.97 but never beyond it. Repeated misses reduce confidence toward 0.12 but never below it. The clamp lets later contradictory observations change a cell instead of allowing confidence to grow without bound.

## 7.3 Ray insertion

For every accepted point, the upstream server determines the sensor origin from TF and transforms the point cloud into the world frame.

When a nonground point is within `sensor_model.max_range`:

- Cells traversed from the origin toward the endpoint receive miss evidence.
- The endpoint cell receives hit evidence.
- A cell observed occupied in the same cloud is not also cleared as free.

When a point lies beyond 15 m:

- The ray is truncated at 15 m.
- Traversed cells receive free-space evidence.
- The truncated endpoint is treated as free, not occupied.

When ground filtering is enabled, segmented ground points clear space but do not create occupied endpoints. This project leaves that filter disabled.

The driver can retain returns as far as its configured 50 m range, while the map integrates only the first 15 m. These are separate controls:

```
driver_range_max = 50.0 m
OctoMap_sensor_model.max_range = 15.0 m
```

The difference limits mapping work and long-range uncertainty without changing the raw cloud available to RViz and bags.

## 7.4 Transform timing

The upstream OctoMap server subscribes through a TF message filter. It requests the transform from the cloud frame to `map` at the cloud header timestamp. The message filter queues a bounded number of clouds and cannot insert a cloud that lacks a valid transform.

The callback also performs a TF lookup with a one-second timeout. If the transform is unavailable, it reports the exception and returns without inserting that cloud.

This gives the core mobile-mapping contract:

```
cloud.header.frame_id = unilidar_lidar
cloud.header.stamp = sensor acquisition time
TF(map -> unilidar_lidar) must resolve at that exact time
```

Publishing only the current pose, using a different ROS domain, omitting a LiDAR extrinsic, or allowing clock discontinuities can all cause cloud drops or geometric distortion.

## 7.5 Stationary and mobile mapping are different contracts

Stationary mode supplies a known identity pose for a fixed bench. Mobile mode supplies no pose and requires an external estimator. They are different engineering contracts, not interchangeable performance settings.

### 7.6 Stationary mode

With `static_sensor:=true`, `l1_octomap.launch.py` starts `static_transform_publisher` with zero translation and rotation.

Stationary mode is suitable for:

- A sensor rigidly fixed to a bench.
- A room or test area observed without moving the sensor base.
- First confirmation that cloud, TF, and mapping interfaces work.
- Testing the map save and view lifecycle.

The L1's internal scanning motion is expected. The restriction concerns motion of the sensor coordinate frame relative to the room.

### 7.7 Mobile mode

With `static_sensor:=false`, the project publishes no substitute pose. External odometry or SLAM must provide the world pose and calibrated frame chain.

A mobile integration must establish:

- A stable world frame and its semantics.
- The LiDAR-to-robot extrinsic transform.
- The LiDAR-to-IMU extrinsic transform if inertial estimation is used.
- Clock alignment among LiDAR, IMU, and pose source.
- A dynamic transform that covers every cloud timestamp.
- Bounded trajectory error appropriate to the mapping task.
- A test for map distortion, scale, drift, and loop behavior.

#### SAFETY / WARNING

Do not switch to mobile mode merely to silence the static-mode warning. `STATIC_SENSOR=false` removes the project TF; it does not create odometry.

## 7.8 Resolution and probability tuning

Resolution controls detail, computation, and memory. Smaller voxels represent finer structure but require more nodes and more observations. Larger voxels reduce cost but merge nearby surfaces and narrow gaps.

Maximum range controls both work and the spatial extent of free-space rays. Increasing it can greatly increase insertion cost and sensitivity to pose and range error.

The hit, miss, and clamp values define how quickly belief changes. They should not be tuned independently of sensor noise, scan overlap, pose quality, and the intended occupancy threshold.

Any parameter experiment should record:

- Configuration commit or patch.
- Dataset or physical layout.
- Sensor mode and TF source.
- Resolution and maximum range.
- Runtime and resource behavior.
- Occupied-node and visible-map results.
- A geometric comparison against an independent reference.

## 7.9 Health check limits

`scripts/evaluate-octomap.sh` subscribes to the transient-local `/occupied_cells_vis_array` and `/octomap_binary` outputs for up to ten seconds. It requires:

- At least one marker array.
- At least one binary map.
- At least one marker containing points.
- A non-empty binary payload.

It reports marker counts, occupied point counts, resolution, payload size, map ID, and frames.

**ENGINEERING NOTE**

OCTOMAP\_MAPPING\_HEALTH\_PASS proves that a non-empty map was published. It does not measure pose error, dimensional accuracy, completeness, noise, map consistency, or safe navigability.

## 8. Hardware preparation and detection

### 8.1 Bench preparation

Before connecting USB or power:

1. Place the L1 on a rigid support.
2. Secure the support against vibration and rotation.
3. Clear the mechanical operating area.
4. Inspect the Unitree cable and adapter.
5. Confirm the supply is 12 V, rated for at least 1 A, and correctly polarized.
6. Disconnect power while making the connections.
7. Connect the adapter to the host USB port.
8. Apply the separate LiDAR supply.

Do not begin software diagnosis until physical power, wiring, and clearance are known.

### 8.2 Observe the host before and after connection

Run:

```
./scripts/check-lidar.sh
```

For the clearest comparison, run it once before connecting the adapter and again afterward.

The script waits for udev, then reports:

- All USB devices from `lsusb`.
- Stable links below `/dev/serial/by-id`.
- Character devices matching `/dev/ttyUSB*` and `/dev/ttyACM*`.
- Resolved device path.
- Mode, owner UID, and group GID.
- USB vendor and product identifiers.
- Vendor, model, serial number, path, and USB driver properties.
- A warning and `fuser` evidence if a process has the device open.

The script makes no changes.

### 8.3 Select one adapter

Automatic runtime detection accepts exactly one unique resolved candidate below `/dev/serial/by-id`.

If several devices are present:

```
export LIDAR_DEVICE="$(
  readlink -e /dev/serial/by-id/<identified-adapter>
)"
test -c "$LIDAR_DEVICE"
export LIDAR_GID="$(stat -Lc '%g' "$LIDAR_DEVICE")"
```

`lidar-launch.sh` resolves the path again and validates the actual character device. It accepts only `ttyUSB*` or `ttyACM*` basenames.

### 8.4 Port ownership

If the port is already open, the launch wrapper prints `fuser -v` evidence and stops. It does not kill the owner.

Possible legitimate owners include:

- An earlier project runtime.
- A manually opened serial terminal.
- Another LiDAR process.
- A device-management service that has positively matched the adapter.

Inspect the process and its purpose. Stop it through its normal control path. Only consider a targeted host rule after repeated evidence tied to the exact VID, PID, and serial number.

## 8.5 Reconnection

After disconnecting and reconnecting:

1. Stop the current launch and recording processes.
2. Check `docker ps` and `docker ps -a`.
3. Run `check-lidar.sh` again.
4. Resolve the stable link again.
5. Start a new container.

Never assume the previous `/dev/ttyUSB0` still represents the L1.

## 9. Canonical live workflow

### 9.1 Prerequisites

Complete the image and workspace build:

```
./scripts/docker-build.sh
./scripts/workspace-build.sh
./scripts/smoke-test.sh
```

For graphical operation, the host session must provide:

```
DISPLAY
XAUTHORITY
/dev/dri/card0
/dev/dri/renderD128
```

For live operation, the L1 must be securely mounted, separately powered, and visible as one selected serial adapter.

### 9.2 First launch without RViz

The first launch removes GUI and rendering from the fault tree.

Terminal 1:

```
./scripts/check-lidar.sh
START_RVIZ=false ./scripts/lidar-launch.sh
```

The wrapper:

1. Validates `START_RVIZ` and `START_MONITOR`.
2. Resolves or selects the device.
3. Refuses a busy or unexpected device.
4. Refuses an existing runtime container name.
5. Derives host UID, GID, and tty GID.
6. Validates the merged Compose configuration.
7. Tests device access in an ephemeral unprivileged container.
8. Creates the named `unitree_l1_runtime` container.
9. Launches the driver and monitor.

The process remains attached to the terminal. Keep it running during validation.

#### EXPECTED RESULT

The preflight prints `LIDAR_CONTAINER_ACCESS_PASS`, followed by driver and monitor launch output.

### 9.3 Require actual messages

Terminal 2:

```
./scripts/lidar-validate.sh
```

The validator runs inside `unitree_l1_runtime` and records its output under:

```
logs/tests/lidar-validation-<timestamp>.log
```

It checks:

- ROS node list.
- Topic list and types.
- Driver `port` parameter.
- Verbose publisher and subscriber information.
- One `PointCloud2` message within 15 seconds.
- One `Imu` message within 15 seconds.
- Measurable average cloud rate.
- Measurable average IMU rate.
- One diagnostic array within 10 seconds.



**EXPECTED RESULT**

Both topics print `MESSAGE_PASS`, both print `RATE_PASS`, and the procedure ends with `LIDAR_DATA_VALIDATION_PASS`.

If the node remains alive but no messages appear, stop with `Ctrl-C`. Recheck power, port identity, ownership, device state, and the known initialization limitation. Do not proceed to RViz or mapping to conceal a failed source.

**9.4 Restart with raw RViz2**

After successful headless validation, stop the first runtime with `Ctrl-C`. Restart it with GUI access:

```
START_RVIZ=true ./scripts/lidar-launch.sh
```

The same wrapper now merges the base, hardware, and GUI Compose files. RViz2 opens with the raw sensor profile.

Use the display to confirm:

- The point cloud updates continuously.
- The shape is stable while the sensor base is stationary.
- The cloud is centered consistently around its sensor frame.
- Gross range or axis errors are absent.
- The display uses `/unilidar/cloud`.
- The fixed frame remains `unilidar_lidar`.

Run in another terminal:

```
./scripts/verify-docker-only.sh
```

The default requires RViz2 and confirms that its process is inside the named container.

**9.5 Add live OctoMap**

Keep the GUI-enabled LiDAR runtime active. Ensure the sensor base will remain stationary.

Terminal 2:

```
STATIC_SENSOR=true \  
OCTOMAP_RVIZ=true \  
./scripts/octomap-launch.sh
```

**SAFETY / WARNING**

`OCTOMAP_RVIZ=true` requires the already-running runtime to have been started with `START_RVIZ=true`. Docker cannot add the X11 cookie or DRI device mounts to a headless container after that container has started.

The mapping wrapper:

- Validates its Boolean settings and runtime name.
- Requires the named container to be running.
- Runs the Docker and GUI assertion inside that container.
- Confirms the installed project package prefix.
- Refuses a second `/octomap_server`.
- Reports whether stationary or mobile mapping was selected.
- Starts OctoMap in the foreground through `docker exec`.

The raw RViz window remains open. `OCTOMAP_RVIZ=true` opens a second window using the map-oriented profile.

**EXPECTED RESULT**

The wrapper prints `OCTOMAP_LAUNCH_READY`, logs the stationary bench warning, and starts `/octomap_server`.

**9.6 Evaluate the live map**

Terminal 3:

```
./scripts/evaluate-octomap.sh
```

For stationary mode, the script identifies `stationary_bench_mapping`. It then waits for non-empty occupied markers and a binary map.

**EXPECTED RESULT**

A successful result begins `OCTOMAP_MAPPING_HEALTH_PASS` and reports non-zero occupied points and binary payload bytes.

If the live cloud is visible but the map is empty, investigate TF before tuning occupancy parameters.

## 9.7 Save a map

While driver and OctoMap remain active:

```
./scripts/save-octomap.sh my_room_01.bt
```

The command requests the current map through the official saver, confines the output to `maps/`, refuses overwrite, and requires a non-empty result.

Inspect it immediately:

```
./scripts/inspect-octomap.sh my_room_01.bt
```

## 9.8 Shutdown

Stop the mapping terminal with `Ctrl-C`, then stop the main driver terminal with `Ctrl-C`.

Check:

```
docker ps
docker ps -a
```

No project runtime should remain unintentionally active. A normal `Compose run --rm` removes the stopped container.

## 9.9 Combined container-side launch

An advanced workflow can start driver, monitor, OctoMap, and one map-profile RViz through the combined launch:

```
./scripts/docker-shell.sh --gui --lidar
```

Inside:

```
ros2 launch ll_octomap_bringup \
  unitree_ll_octomap.launch.py \
  static_sensor:=true \
  rviz:=true
```

The combined launch includes both project launch files in scoped groups. It forces the child RViz arguments to false and owns one public `rviz` argument, preventing duplicate RViz processes.

Use the host wrappers for normal operation because they provide device detection, busy-port checks, access preflight, named-container controls, and consistent diagnostics.

# 10. Recording and replay

## 10.1 Recording contract

`scripts/record-bag.sh` records from the active named runtime. Its defaults are:

| Setting                        | Default                         |
|--------------------------------|---------------------------------|
| <code>RUNTIME_CONTAINER</code> | <code>unitree_ll_runtime</code> |
| <code>BAG_LABEL</code>         | <code>validation</code>         |
| <code>BAG_DURATION_SEC</code>  | <code>0</code>                  |

The label must begin with an alphanumeric character and may contain alphanumerics, underscore, period, and hyphen. Duration is a whole number of seconds. Zero means operator-controlled recording.

The output name is:

```
bags/ll_<label>_<YYYYMMDD_HHMMSS>
```

## 10.2 Topic selection

The wrapper refuses to record until it receives one message on each mandatory topic:

```
/unilidar/cloud
/unilidar/imu
```

It always records those topics. At recording start, it also includes each available optional topic:

```
/diagnostics
/tf
/tf_static
```

The standard wrapper does not include:

- `/occupied_cells_vis_array`.
- `/octomap_binary` or `/octomap_full`.
- `/projected_map`.
- Arbitrary odometry topics.
- Application-specific commands or state.

If a future mobile dataset needs another topic, change the wrapper through normal review and validate the resulting metadata. Do not assume `/tf` alone captures every estimator input needed for reproducibility.

## 10.3 Bounded recording

For a 30-second validation sample:

```
BAG_LABEL=validation \
BAG_DURATION_SEC=30 \
./scripts/record-bag.sh
```

The wrapper sends `SIGINT` at the duration and allows up to ten additional seconds for clean finalization. It accepts the expected timeout exit and then runs `ros2 bag info`.

### EXPECTED RESULT

`metadata.yaml` exists and both mandatory topic counts are greater than zero.

For an operator-controlled recording:

```
BAG_LABEL=mapping \
BAG_DURATION_SEC=0 \
./scripts/record-bag.sh
```

Stop it with `Ctrl-C`.

## 10.4 Bag inspection

Use a path below the project `bags/` directory:

```
./scripts/bag-info.sh \
bags/l1_validation_<timestamp>
```

The script resolves the real path, rejects paths outside `bags/`, requires `metadata.yaml`, converts the path to `/workspace/bags/...`, and runs `ros2 bag info` in a clean container.

Review:

- Storage identifier.
- Duration.
- Start and end time.
- Topic names and types.
- Message counts.
- Serialized format.
- Individual database files.

## 10.5 Replay

Stop the hardware runtime before replay:

```
START_RVIZ=true \
./scripts/replay-bag.sh \
bags/l1_validation_<timestamp>
```

The replay wrapper needs no LiDAR overlay. It optionally adds the GUI overlay, starts `l1_monitor`, and starts raw RViz2. Both use `use_sim_time:=true`. After a two-second startup interval it runs:

```
ros2 bag play <bag> --clock
```

The cleanup trap terminates and waits for monitor and RViz processes when playback ends.

Replay is intended to inspect the raw sensor contract. It does not automatically launch OctoMap. A reproducible mapping replay would also need the proper TF stream, mapping configuration, and an explicitly coordinated OctoMap process using simulated time.

## 10.6 Data retention

Bags may be large and can contain details of a physical location. They are ignored by Git. Establish external retention, privacy, backup, and deletion rules appropriate to the deployment.

Do not use a validation bag as an undocumented permanent reference. Record its purpose, hardware state, project commit, and checks alongside the protected dataset.

# 11. Map lifecycle

## 11.1 Save behavior

`scripts/save-octomap.sh` accepts a safe basename ending in `.bt` or `.ot`. With no argument, it creates:

```
l1_octomap_<YYYYMMDD_HHMMSS>.bt
```

The script:

1. Validates the runtime container name.
2. Validates the map basename and extension.
3. Confines the host path to `maps/`.
4. Requires the runtime container.

5. Creates `maps/` on demand.
6. Refuses an existing output.
7. Requires the `/octomap_binary` service.
8. Runs `octomap_saver_node`.
9. Requires a non-empty host file.

The saver is invoked with `octomap_path` but without `full:=true`. It therefore requests the binary map service for both filename extensions. A `.bt` output uses OctoMap binary serialization. A `.ot` output uses the full file container format on the tree reconstructed from that binary service response.

#### ENGINEERING NOTE

Do not claim that `.ot` created by this wrapper preserves the full live probabilistic state. The wrapper's service request is binary unless it is deliberately changed and revalidated.

## 11.2 Structural inspection

Run:

```
./scripts/inspect-octomap.sh my_room_01.bt
```

The inspector requires:

- A safe `.bt` or `.ot` basename.
- A readable file below `maps/`.
- A first line beginning with `# Octomap`.
- A valid alphanumeric tree ID.
- A positive integer stored-node count.
- A non-negative decimal resolution field.
- A header ending with the `data` marker.

It reports:

```
absolute file path
format
tree ID
stored nodes
resolution in metres
file bytes
modification time
SHA-256
```

This is a bounded structural check. It does not deserialize and traverse every node, compare the map with the environment, or prove that binary payload bytes are free from all corruption.

## 11.3 Reopen a saved map

After stopping live mapping:

```
./scripts/view-octomap.sh my_room_01.bt
```

The wrapper first calls the inspector. It then starts a dedicated Compose container with defaults:

| Setting                  | Default               |
|--------------------------|-----------------------|
| MAP_VIEWER_CONTAINER     | unitree_l1_map_viewer |
| MAP_VIEWER_ROS_DOMAIN_ID | 43                    |
| MAP_VIEWER_RVIZ          | true                  |

The viewer domain must be an integer from 0 to 232. Domain 43 separates saved map inspection from the normal domain-42 runtime.

The launch loads the map through the upstream `octomap_path` parameter and sets both `frame_id` and `base_frame_id` to `map`. The saved RViz profile subscribes to the transient-local occupied marker array.

Press `Ctrl-C` to close server and RViz together.

## 11.4 Headless checks

Validate a map and viewer configuration without starting the viewer:

```
MAP_VIEWER_RVIZ=false \
./scripts/view-octomap.sh my_room_01.bt --check
```

Validate a live OctoMap launch surface without starting OctoMap:

```
./scripts/octomap-launch.sh --check
```

The latter still requires a running driver container because it verifies the installed package and existing ROS graph.

## 11.5 Map provenance

For an engineering map record, retain outside Git:

- Map filename and SHA-256.
- Source bag or live session identifier.
- Project commit and dependency commits.
- OctoMap parameter set.
- Mapping mode and TF source.
- Sensor mounting and extrinsics.
- Room or environment identifier.
- Start and end time.
- Operator.
- Validation results and known limitations.

The repository intentionally stores the reproducible procedures, not the generated map.

## 12. Diagnostics and operational observability

### 12.1 Observe the graph

Run observations inside the named runtime through the supplied validators or an interactive container. Do not install or run a parallel host ROS graph.

Useful commands inside the container are:

```
ros2 node list
ros2 topic list -t
ros2 service list
ros2 param list /unitree_lidar_ros2_node
ros2 param get /unitree_lidar_ros2_node port
```

For topic endpoints:

```
ros2 topic info -v /unilidar/cloud
ros2 topic info -v /unilidar/imu
ros2 topic info -v /diagnostics
```

For one message:

```
ros2 topic echo /unilidar/cloud \
  sensor_msgs/msg/PointCloud2 \
  --once

ros2 topic echo /unilidar/imu \
  sensor_msgs/msg/Imu \
  --once
```

For measured rates:

```
ros2 topic hz /unilidar/cloud
ros2 topic hz /unilidar/imu
```

Stop continuous inspection commands with `Ctrl-C`.

### 12.2 Interpret /diagnostics

Read one diagnostic array:

```
ros2 topic echo /diagnostics \
  diagnostic_msgs/msg/DiagnosticArray \
  --once
```

An OK message means the monitor observed data recently, measured a rate above its configured minimum, saw no non-increasing timestamps, and found no current header-age warning.

A WARN is actionable even when data remains visible. It can indicate:

- Startup before the first message.
- Frequency below the provisional threshold.
- One or more zero timestamps.
- Header time more than one second away from the ROS clock.

An ERROR indicates:

- No data after the startup timeout.
- A stream that was active but is now stale.
- At least one non-increasing timestamp.

The monitor counters are cumulative for the lifetime of the node. A timestamp error does not clear itself from the status merely because later messages are correct. Restart the monitor after the underlying issue is understood if a fresh measurement window is required.

## 12.3 Rate interpretation

The monitor calculates a moving-window arrival rate, while `ros2 topic hz` measures from its own subscription. Small differences are normal because the windows, startup time, scheduling, and DDS delivery differ.

The current warning thresholds of 5 Hz for cloud and 20 Hz for IMU are lower than validated normal operation. They are designed to detect severe degradation rather than enforce the manufacturer's nominal rate.

If the cloud falls from approximately 8 to 10 Hz toward the 5 Hz threshold:

- Confirm CPU and rendering load.
- Compare headless and graphical operation.
- Check USB and serial ownership.
- Inspect message timestamp progression.
- Determine whether point counts or cloud aggregation changed.
- Record a bounded bag for repeatable analysis.

## 12.4 Point metadata

The diagnostic cloud status exposes the latest `width * height` and field names. This is useful for detecting a source that publishes the right ROS type but the wrong schema.

The expected fields from the pinned driver are:

```
x,y,z,intensity,ring,time
```

Do not infer unit correctness from the field names alone. Confirm units and extrinsics before choosing or tuning an external estimator.

## 12.5 TF observation

For stationary mapping, inspect the transform:

```
ros2 run tf2_ros tf2_echo \
  map \
  unilidar_lidar
```

The expected stationary translation and rotation are zero.

For mobile mapping, observe that the transform changes with motion and remains available at cloud times. A current transform in `tf2_echo` is necessary but not sufficient; the TF buffer must also cover historical cloud timestamps.

Inspect published TF topics:

```
ros2 topic info -v /tf
ros2 topic info -v /tf_static
```

## 12.6 Docker process observation

From the host:

```
docker ps
docker top unitree_l1_runtime
docker inspect unitree_l1_runtime
```

Use `verify-docker-only.sh` for the bounded project check:

```
REQUIRE_RVIZ=true \
./scripts/verify-docker-only.sh
```

If the runtime intentionally has no RViz:

```
REQUIRE_RVIZ=false \
./scripts/verify-docker-only.sh
```

The script reports host role, host OS, container image, container PID, Docker runtime assertion, and the final pipeline verdict. It does not run a host ROS command.

## 12.7 Local logs

`lidar-validate.sh` creates timestamped files below `logs/tests/`. Colcon creates its own logs below `ros2_ws/log/`. Docker and ROS launch also print to the attached terminal.

These paths are generated evidence and remain ignored. Preserve a required record in controlled external storage, with its project commit and context. Do not commit raw logs merely to prove that a past command ran.

## 13. Verification and acceptance

### 13.1 Validation philosophy

Validation proceeds from the least coupled layer to the most coupled:

```
repository -> Compose -> dependencies -> build -> project tests
-> runtime -> monitor -> GUI -> serial access -> live data
-> recording -> stationary mapping -> map lifecycle
```

Do not skip a failed lower layer and attempt to compensate at a higher layer. For example, RViz cannot repair an absent cloud, and OctoMap parameters cannot repair missing TF.

### 13.2 Compose validation

The base, GUI, and LiDAR overlays must merge successfully. The runtime wrappers call `docker compose ... config --quiet` before starting their main process.

The GUI merge requires valid host values for:

```
DISPLAY
XAUTHORITY
VIDEO_GID
RENDER_GID
```

The hardware merge requires:

```
LIDAR_DEVICE
LIDAR_GID
```

### 13.3 Dependency lock validation

Run:

```
./scripts/fetch-dependencies.sh
```

#### EXPECTED RESULT

Unitree and OctoMap each report `dependency ready` at the exact configured commit, with no dirty checkout.

### 13.4 Package and output validation

Inside a headless project shell:

```
cd /workspace/ros2_ws
colcon list
```

Exactly six intended packages should appear.

The output-isolation test may invoke `colcon` from `/workspace/ros2_ws/src`; the configured absolute bases must still place all output under `/workspace/ros2_ws`.

From the host, this command must produce no output:

```
find ros2_ws/src -type d \
  \( -name build -o -name install -o -name log \) \
  -print
```

### 13.5 Workspace build validation

Run:

```
./scripts/workspace-build.sh
```

All six packages must finish without failure. The source-tree postcondition must remain clean.

Then:

```
./scripts/smoke-test.sh
```

The smoke test confirms:

- Ubuntu 22.04 and ROS 2 Humble.
- `ros2`, `colcon`, and `RViz2`.
- Installed Unitree driver package and executable.
- Installed monitor executable.
- Project launch argument parsing.
- Unitree executable dynamic-library resolution.
- PointCloud2 interface availability.

#### EXPECTED RESULT

The command ends with `SMOKE_TEST_PASS`.

## 13.6 Project unit and contract tests

Run:

```
./scripts/docker-shell.sh --no-gui --no-lidar
```

Inside:

```
cd /workspace/ros2_ws
colcon test \
  --packages-select l1_bringup l1_monitor l1_octomap_bringup
colcon test-result --all --verbose
```

The current suite contains:

| Package            | Tests | Coverage focus                       |
|--------------------|-------|--------------------------------------|
| l1_bringup         | 3     | Launch contract and Python style     |
| l1_monitor         | 5     | Statistics, errors, and Python style |
| l1_octomap_bringup | 9     | Launch, RViz, metadata, and style    |

### EXPECTED RESULT

17 tests, zero errors, zero failures, and zero skipped tests.

## 13.7 Synthetic monitor validation

Run:

```
./scripts/monitor-synthetic-test.sh
```

The test uses ROS domain 187 by default and `ROS_LOCALHOST_ONLY=1` to avoid contamination from other graphs. It starts:

- The monitor with a 0.5-second report period.
- A 10 Hz four-point synthetic PointCloud2 publisher.
- A 30 Hz synthetic Imu publisher.

It requires both diagnostic messages to be `stream healthy`, point count four, and fields `x`, `y`, `z`.

### EXPECTED RESULT

`MONITOR_SYNTHETIC_HEALTH_PASS`.

## 13.8 GUI validation

Run from the active graphical host session:

```
./scripts/gui-smoke-test.sh
```

It validates the X display, OpenGL renderer, RViz2 executable, Docker runtime, and GUI device boundary.

### EXPECTED RESULT

`GUI_SMOKE_TEST_PASS`.

## 13.9 Hardware data validation

With the headless driver runtime active:

```
./scripts/lidar-validate.sh
```

Acceptance requires correct types, at least one message on each mandatory topic, measurable rates, and one diagnostic message.

The validated reference range is:

| Stream      | Measured                    | Official initial reference |
|-------------|-----------------------------|----------------------------|
| Point cloud | approximately 8 to 10 Hz    | approximately 11 Hz        |
| IMU         | approximately 210 to 250 Hz | approximately 250 Hz       |

Do not convert those observations into hard manufacturer guarantees.

## 13.10 Recording validation

Record a bounded bag:

```
BAG_LABEL=validation \
BAG_DURATION_SEC=30 \
./scripts/record-bag.sh
```

Inspect it:

```
./scripts/bag-info.sh \
bags/l1_validation_<timestamp>
```

Cloud and IMU counts must both be positive. Replay must exit cleanly after the recorded duration.



## 13.11 Stationary mapping validation

With a stationary L1 and valid runtime:

```
STATIC_SENSOR=true \
OCTOMAP_RVIZ=false \
./scripts/octomap-launch.sh
```

In another terminal:

```
./scripts/evaluate-octomap.sh
./scripts/save-octomap.sh validation.bt
./scripts/inspect-octomap.sh validation.bt
```

Acceptance requires a non-empty binary map, non-empty occupied markers, a non-empty saved file, a valid header, and positive stored-node count.

## 13.12 Mobile mapping acceptance

Mobile mapping is pending and cannot inherit the stationary acceptance. A future validation must include:

- Dynamic TF coverage at every cloud timestamp.
- Verified LiDAR, IMU, and base extrinsics.
- Defined world, odometry, and base-frame semantics.
- Trajectory accuracy against an independent reference.
- Bounded drift.
- Map scale and dimensional checks.
- Repeatability across runs.
- Failure behavior when TF or estimator data is lost.

## 13.13 Repository hygiene

Run:

```
git diff --check

find ros2_ws/src -type d \
  \( -name build -o -name install -o -name log \) \
  -print

git ls-files | grep -E \
  '(^|/)(build|install|log|__pycache__) (/|$)|\.pyc$'
```

The two discovery commands must print nothing. A valid local build may leave ignored output at `ros2_ws/build`, `ros2_ws/install`, and `ros2_ws/log`. `git status` must not acquire generated files.

# 14. Troubleshooting

## 14.1 Troubleshooting method

Work from physical cause toward application behavior:

1. Confirm safety, power, cable, and adapter.
2. Confirm host device detection and stable identity.
3. Confirm device ownership and access.
4. Confirm container composition and Docker assertion.
5. Confirm driver process and configured port.
6. Confirm one real message on each topic.
7. Confirm rates, timestamps, frames, and fields.
8. Confirm GUI separately.
9. Confirm TF before OctoMap.
10. Confirm map outputs before saving or judging quality.

Capture the exact command, complete error, project commit, and changed conditions. Avoid speculative system changes.

## 14.2 No serial device appears

Run the discovery script once with the L1 disconnected and again after the adapter and separate power supply have been connected safely:

```
./scripts/check-lidar.sh
```

If no stable link or tty appears, check 12 V power, polarity, adapter, cable, connector, and USB port.

Do not invent a device path, create a substitute symlink, broaden permissions, or install a host ROS driver. The launcher cannot recover from a device that the Linux host has not enumerated.

### 14.3 Several serial devices are detected

Automatic launch deliberately stops unless exactly one stable `/dev/serial/by-id` candidate resolves to a character device. This prevents a changing tty number from selecting unrelated hardware.

Use the before-and-after `check-lidar.sh` reports to identify the new adapter. Compare `ID_VENDOR_ID`, `ID_MODEL_ID`, `ID_SERIAL_SHORT`, and `ID_PATH`; prefer the stable by-id link when the adapter supplies one. Resolve the chosen link before exporting it:

```
export LIDAR_DEVICE="$(
  readlink -e /dev/serial/by-id/<identified-adapter>
)"
test -c "$LIDAR_DEVICE"
```

Keep this choice local. Do not commit a machine-specific identifier or select the lowest `ttUSB*` number merely because it appears first.

### 14.4 The selected serial path is rejected

The launcher accepts an explicit path only when it resolves, names a character device, and ends at a `ttUSB*` or `ttACM*` device. Inspect the exact value:

```
printf 'LIDAR_DEVICE=%s\n' "${LIDAR_DEVICE:-<unset>}"
readlink -e -- "${LIDAR_DEVICE:-/nonexistent}"
test -n "${LIDAR_DEVICE:-}" && test -c "$LIDAR_DEVICE"
```

A blank `readlink -e` result means the link is missing or stale; a failed character-device test means it is not a usable tty. Reconnect only with L1 power removed, rerun discovery, and export the new identity. Never bypass the basename check to expose a regular file, pseudo-terminal, or unrelated device.

### 14.5 The serial device is already open

Both the discovery and launch scripts use `fuser` when available. The launcher prints the owning process information and exits without stopping anything. Repeat the read-only checks and inspect project containers:

```
./scripts/check-lidar.sh
fuser -v "$LIDAR_DEVICE" || true
docker ps -a --filter name=unitree_l1
```

Identify the process before acting. If it is the intended live project run, return to its terminal and stop it with `Ctrl-C`. If it belongs to another application or service, follow that owner's normal shutdown procedure only when authorized. Never use a blanket process kill, disable an unknown service, or change device permissions to compete for an already-open port.

### 14.6 The runtime container name already exists

The default runtime name is `unitree_l1_runtime`. Before creating a new live run, the wrapper checks Docker for any container with that exact name and refuses to overwrite it.

```
docker ps -a \
  --filter name=unitree_l1_runtime \
  --format 'table {{.Names}}\t{{.Status}}\t{{.Image}}'
docker inspect unitree_l1_runtime \
  --format 'status={{.State.Status}} image={{.Config.Image}}' 2>/dev/null || true
```

For an active supported run, stop the launch from its original terminal with `Ctrl-C`; Compose used `run --rm`, so a clean exit removes the temporary container. If an exited container remains after an abnormal interruption, confirm its identity and retain any needed logs before removing that exact container through the normal Docker workflow. Changing `RUNTIME_CONTAINER` can support an intentional second isolated run, but it must not be used to hide an unexplained stale runtime or to make two processes contend for one LiDAR.

## 14.7 Container serial access fails

The hardware overlay maps the resolved host tty to `/dev/unitree_lidar:rwm`. `lidar-launch.sh` reads the host tty group ID, supplies it as a supplementary container group, and runs a preflight that requires the mapped device to be a readable and writable character device.

Collect the host-side identity and access facts:

```
stat -Lc 'path=%n mode=%A uid=%u gid=%g' "$LIDAR_DEVICE"
id
./scripts/check-lidar.sh
```

Then rerun the supported launcher and retain the complete preflight output. A successful boundary check prints:

```
container_device=/dev/unitree_lidar ...
LIDAR_CONTAINER_ACCESS_PASS
```

If it fails, verify that `LIDAR_DEVICE` still resolves to the same tty and that no process owns it. Recreate the container through `lidar-launch.sh` so the current tty GID is recomputed; group and device mappings cannot be added to an already-created container. Never use `chmod 777`, privileged mode, global udev rules, or host ROS as a repair.

## 14.8 Driver node exists but topics are silent

This is a known dangerous ambiguity because the upstream ROS 2 driver ignores the initialization return.

Check:

```
ros2 param get /unitree_lidar_ros2_node port
ros2 topic info -v /unilidar/cloud
ros2 topic info -v /unilidar/imu
```

Then require messages:

```
timeout 15s \
ros2 topic echo /unilidar/cloud --once

timeout 15s \
ros2 topic echo /unilidar/imu --once
```

If both remain silent:

- Stop cleanly.
- Recheck power and adapter identity.
- Recheck port ownership.
- Consider whether the L1 is in normal working mode.
- Preserve evidence before proposing project-owned driver correction.

## 14.9 One stream is present and the other is absent

The same serial parser produces both streams. A partial result can indicate a parser, message-type, rate, or sensor-state issue.

Capture:

- Topic endpoint details.
- One available message.
- Diagnostic status for both streams.
- Driver terminal output.
- Short bag if recording requirements can be met.
- Firmware and SDK versions if exposed through controlled diagnostics.

Do not lower the monitor threshold to convert absence into success.

## 14.10 Frequency is low

Compare headless and GUI runs. RViz and OctoMap add CPU, GPU, DDS, and conversion work.

Check:

```
ros2 topic hz /unilidar/cloud
ros2 topic hz /unilidar/imu
docker stats unitree_l1_runtime
```

Investigate scheduling load, USB behavior, point count, cloud aggregation, rendering, and mapping load. Preserve the configuration and measured interval.

## 14.11 Timestamp diagnostics fail

Zero stamps cause WARN. Non-increasing stamps cause ERROR. Excessive positive or negative header age causes WARN.

Inspect raw headers and the current ROS time:

```
ros2 topic echo /unilidar/cloud --once
ros2 topic echo /unilidar/imu --once
ros2 topic echo /clock --once
```

/clock normally exists during bag replay with --clock, not ordinary live operation. Check for mixed use of simulated and system time.

## 14.12 RViz does not open

Typical causes are:

- Missing DISPLAY.
- Missing or unreadable XAUTHORITY.
- Absent /dev/dri/card0.
- Absent /dev/dri/renderD128.
- Wrong video or render GID.
- A graphical session that changed after environment export.

Run ./scripts/gui-smoke-test.sh.

Use --no-gui or START\_RVIZ=false for headless work rather than weakening X11 security.

## 14.13 Raw RViz opens but the cloud is blank

Confirm:

```
Fixed Frame = unilidar_lidar
Topic = /unilidar/cloud
Reliability = Reliable
Durability = Volatile
```

Then:

```
ros2 topic echo /unilidar/cloud --once
ros2 topic info -v /unilidar/cloud
```

If data exists, inspect RViz status messages, display enablement, camera range, and point size. Do not change the fixed frame to map unless a valid transform exists.

## 14.14 OCTOMAP\_RVIZ=true fails after headless startup

The existing container lacks GUI mounts. Docker Compose overlays are applied when the container is created, not dynamically.

Stop the headless runtime and restart:

```
START_RVIZ=true ./scripts/lidar-launch.sh
```

Then, in another terminal:

```
STATIC_SENSOR=true \
OCTOMAP_RVIZ=true \
./scripts/octomap-launch.sh
```

### SAFETY / WARNING

Setting only OCTOMAP\_RVIZ=true cannot retrofit X11 or DRI into the already-running headless runtime.

## 14.15 OctoMap is already running

The wrapper refuses a second /octomap\_server in the same ROS graph.

Check:

```
ros2 node list
```

Stop the first mapping process cleanly. Do not run competing servers with the same names and topics in the standard graph.

## 14.16 Cloud is visible but OctoMap is empty

TF is the first suspect.

Check:

```
ros2 node list
ros2 topic info -v /unilidar/cloud
ros2 run tf2_ros tf2_echo \
  map \
  unilidar_lidar
```

Also inspect `/octomap_server` output for transform exceptions. Ensure cloud header frame and launch `lidar_frame` agree.

If stationary mode is active, `/l1_static_lidar_transform` should exist. If mobile mode is active, it must not exist, and an external dynamic transform must resolve.

### 14.17 OctoMap appears smeared or duplicated

Stop mapping before saving. Common causes are:

- The sensor moved while stationary identity TF was active.
- Dynamic pose is delayed or unavailable at cloud timestamps.
- Extrinsic transform is wrong.
- World-frame semantics changed.
- Pose scale or orientation is wrong.
- Cloud timestamps are discontinuous.

Discard a known false map. Do not tune hit probability or resolution to hide a pose error.

### 14.18 Remaining fault index

| Symptom                            | Required check                                   | Correct response                                |
|------------------------------------|--------------------------------------------------|-------------------------------------------------|
| Map too sparse or dense            | Source, TF, range, sensor model                  | Tune one variable against a reference           |
| Save fails                         | Runtime, safe name, <code>/octomap_binary</code> | Keep existing files; choose a new name          |
| Inspection fails                   | Header, tree ID, nodes, resolution               | Save again; compare checksums                   |
| Viewer fails                       | Map check, name, domain, GUI                     | Use <code>view-octomap.sh MAP --check</code>    |
| Recorder refuses                   | Cloud and IMU messages                           | Run <code>lidar-validate.sh</code> ; fix source |
| Bag path rejected                  | Directory below <code>bags/</code> , metadata    | Preserve the complete bag directory             |
| Replay time stalls                 | Bag duration, playback, <code>/clock</code>      | Keep replay nodes on simulated time             |
| Dependency check fails             | Commit and clean worktree                        | Preserve work; deliberately recreate            |
| Output appears in <code>src</code> | Recursive output search                          | Remove exact output; use build wrapper          |
| ROS graph is split                 | <code>ROS_DOMAIN_ID</code> for each process      | Give communicating nodes one domain             |

Do not use occupancy tuning to compensate for TF or source failure. Do not overwrite a map, bypass bag path confinement, discard unreviewed vendor changes, or remove a directory until its generated origin is confirmed.

## 15. Maintenance and change control

### 15.1 Ownership rules

Make changes in the layer that owns the behavior:

| Change                              | Correct owner                          |
|-------------------------------------|----------------------------------------|
| Driver launch or project parameters | <code>l1_bringup</code>                |
| Stream status logic                 | <code>l1_monitor</code>                |
| OctoMap remap, TF mode, or profile  | <code>l1_octomap_bringup</code>        |
| Host workflow and safety checks     | <code>scripts/</code>                  |
| Container packages or boundary      | <code>docker/</code>                   |
| External source version             | <code>config/dependencies.repos</code> |
| Architecture rationale              | <code>docs/decisions.md</code>         |
| Acceptance evidence                 | <code>docs/validation-matrix.md</code> |

Do not modify generated install copies. Do not edit ignored vendor source as the permanent solution.

### 15.2 Change classification

A change requires hardware revalidation when it affects:

- Unitree SDK or driver commit.
- Serial port handling or baud rate.
- Initialization or working mode.
- Range calibration.
- Cloud aggregation.

- Topic type, name, frame, or QoS.
- Timestamp handling.
- Point fields or units.
- IMU content.
- TF topology or extrinsics.
- OctoMap resolution, range, sensor model, or filtering.
- Docker device or group controls.

A documentation-only change still requires renderer and link validation, but does not automatically require powered hardware.

### 15.3 Dependency update procedure

For a deliberate upstream update:

1. Review upstream release notes, license, and supported environment.
2. Record the candidate immutable commit.
3. Update `config/dependencies.repos`.
4. Recreate or deliberately advance the ignored checkout.
5. Require a clean checkout at the new commit.
6. Rebuild the Docker image if system dependencies changed.
7. Build all six intended packages.
8. Run all project tests and software smoke tests.
9. Inspect topic, field, QoS, timestamp, and frame contracts.
10. Repeat the complete hardware validation.
11. Repeat recording, replay, and stationary mapping.
12. Update versions, sources, decisions, and validation records.

Do not call a dependency update complete because compilation succeeded.

### 15.4 Project parameter changes

For a parameter experiment:

1. State the problem and measurable acceptance criterion.
2. Preserve the baseline configuration and result.
3. Change the smallest relevant parameter set.
4. Rebuild or re-source as required.
5. Run the applicable automated tests.
6. Run a controlled dataset or physical test.
7. Compare against the baseline.
8. Record side effects and limitations.
9. Commit configuration and documentation together.

The symlink install supports iteration, but a clean workspace build remains the release acceptance path.

### 15.5 Monitor threshold changes

The current thresholds are operational alarms. Before changing them, measure the normal distribution of rates and timestamp age across:

- Headless driver operation.
- Raw RViz operation.
- Live OctoMap operation.
- Bag replay.
- Expected host load.
- Reconnect and startup behavior.

Set thresholds to detect meaningful degradation without normalizing a known fault. Update `docs/decisions.md` when the rationale changes.

### 15.6 Mobile mapping integration

A future estimator integration should remain a separate project layer. It must not turn OctoMap into an implicit pose estimator.

The integration review must define:

- Selected estimator repository and immutable commit.
- Supported ROS distribution.
- Point field assumptions.
- IMU units and covariance assumptions.

- Timestamp and synchronization policy.
- LiDAR-to-IMU and base extrinsics.
- World, odometry, and base frame conventions.
- TF publication authority.
- Initialization and loss-of-tracking behavior.
- Resource requirements.
- Recording and replay topic set.
- Accuracy and drift acceptance tests.

## 15.7 Docker maintenance

When adding a runtime dependency:

- Add the package to `docker/Dockerfile`.
- Retain `--no-install-recommends` unless a required dependency proves

otherwise.

- Keep the image user unprivileged.
- Keep build context narrow.
- Rebuild with `docker-build.sh`.
- Repeat Docker assertion, smoke, GUI, and hardware checks as applicable.

Do not add host networking, a broad device mapping, a capability, or `privileged: true` without a documented threat and alternatives analysis.

## 15.8 Generated-data policy

The following are intentionally local:

| Path                              | Producer                            |
|-----------------------------------|-------------------------------------|
| <code>ros2_ws/build/</code>       | colcon build and test               |
| <code>ros2_ws/install/</code>     | colcon build                        |
| <code>ros2_ws/log/</code>         | colcon verbs                        |
| <code>bags/</code>                | bag recorder                        |
| <code>maps/</code>                | OctoMap saver                       |
| <code>logs/</code>                | validation wrappers                 |
| <code>exports/manuals/</code>     | three reviewed, tracked manual PDFs |
| Other <code>exports/</code> paths | ignored optional exports            |

Python bytecode, test caches, editor state, local environment files, X11 authorization files, logs, archives, and non-manual exports are ignored. The three named project manual PDFs are tracked for direct GitHub access.

Generated evidence may need controlled external retention. Ignored does not mean unimportant; it means not suitable for the source repository.

## 15.9 Documentation maintenance

Update prose when executable behavior changes. Keep commands repository relative and test them from the repository root.

When changing a workflow:

- Update its source script.
- Update this manual and the concise README if applicable.
- Update the decision log if the architecture changed.
- Update the validation matrix with a reproducible command.
- Regenerate the PDF manuals.
- Inspect page size, text, tables, code blocks, and callouts.

## 15.10 Release acceptance

A project release should not be declared until:

- Git contains only intended source changes.
- Dependency commits are correct and clean.
- Compose overlays validate.
- The image builds.
- Six packages are discovered and built.
- All project tests pass.
- Runtime and synthetic monitor smoke tests pass.
- GUI validation passes when GUI behavior changed.
- Hardware validation passes when hardware behavior changed.

- Stationary mapping and map lifecycle pass when mapping changed.
- Documentation matches the release commit.

## 16. Environment variable reference

### 16.1 Common runtime variables

| Variable          | Default            | Used by           | Purpose              |
|-------------------|--------------------|-------------------|----------------------|
| ROS_DOMAIN_ID     | 42                 | Compose           | ROS graph isolation  |
| RUNTIME_CONTAINER | unitree_l1_runtime | Runtime scripts   | Named live container |
| LIDAR_DEVICE      | Auto               | Shell and launch  | Explicit host tty    |
| START_RVIZ        | Script-specific    | Launch and replay | Start raw RViz       |
| START_MONITOR     | true               | LiDAR launch      | Start monitor        |

RUNTIME\_CONTAINER must begin with an alphanumeric character and contain only alphanumerics, underscore, period, and hyphen.

### 16.2 GUI variables

| Variable    | Source          | Purpose             |
|-------------|-----------------|---------------------|
| DISPLAY     | Host session    | X11 display         |
| XAUTHORITY  | Host session    | X11 cookie file     |
| VIDEO_GID   | Wrapper derives | DRI card group      |
| RENDER_GID  | Wrapper derives | DRI render group    |
| REQUIRE_GUI | Wrapper sets    | Container assertion |

Operators normally set only DISPLAY and XAUTHORITY. The wrappers derive device GIDs from the active host.

### 16.3 LiDAR variables

| Variable     | Default                       | Purpose                       |
|--------------|-------------------------------|-------------------------------|
| LIDAR_DEVICE | Auto-detect one stable device | Host tty selection            |
| LIDAR_GID    | Derived                       | Supplementary container group |
| LIDAR_PORT   | /dev/unitree_lidar            | Container driver path         |

LIDAR\_PORT is set by the Compose LiDAR overlay. Do not normally export it on the host.

### 16.4 OctoMap variables

| Variable                 | Default               | Purpose                     |
|--------------------------|-----------------------|-----------------------------|
| STATIC_SENSOR            | true                  | Bench identity TF selection |
| OCTOMAP_RVIZ             | false                 | Start live map RViz         |
| MAP_VIEWER_CONTAINER     | unitree_l1_map_viewer | Saved viewer name           |
| MAP_VIEWER_ROS_DOMAIN_ID | 43                    | Saved viewer isolation      |
| MAP_VIEWER_RVIZ          | true                  | Start saved-map RViz        |

#### SAFETY / WARNING

OCTOMAP\_RVIZ=true works only when the existing live runtime was created by START\_RVIZ=true  
./scripts/lidar-launch.sh.

### 16.5 Bag variables

| Variable         | Default    | Constraint          |
|------------------|------------|---------------------|
| BAG_LABEL        | validation | Safe basename token |
| BAG_DURATION_SEC | 0          | Whole seconds       |

Zero duration means record until Ctrl-C.

### 16.6 Validation variables

| Variable                   | Default | Purpose                   |
|----------------------------|---------|---------------------------|
| REQUIRE_RVIZ               | true    | Docker-only process check |
| MONITOR_TEST_ROS_DOMAIN_ID | 187     | Synthetic test isolation  |

ROS domain IDs accepted by project validation are integers from 0 through 232.



## 16.7 Build variables

| Variable             | Source          | Purpose                |
|----------------------|-----------------|------------------------|
| HOST_UID             | Wrapper derives | Image user ID          |
| HOST_GID             | Wrapper derives | Image primary group    |
| COLCON_DEFAULTS_FILE | Compose sets    | Canonical colcon paths |

The defaults file path is:

```
/workspace/ros2_ws/colcon_defaults.yaml
```

## 17. ROS interface inventory

### 17.1 Nodes

| Node                       | Package            | Mode              |
|----------------------------|--------------------|-------------------|
| /unitree_lidar_ros2_node   | unitree_lidar_ros2 | Live              |
| /l1_monitor                | l1_monitor         | Live or replay    |
| /rviz2                     | rviz2              | Raw or combined   |
| /octomap_server            | octomap_server     | Live or saved     |
| /l1_static_lidar_transform | tf2_ros            | Stationary only   |
| /octomap_rviz2             | rviz2              | Separate live map |
| /saved_octomap_rviz2       | rviz2              | Saved map         |

### 17.2 Sensor and diagnostic topics

| Topic           | Type                                | Publisher      |
|-----------------|-------------------------------------|----------------|
| /unilidar/cloud | sensor_msgs/msg/PointCloud2         | Unitree driver |
| /unilidar/imu   | sensor_msgs/msg/Imu                 | Unitree driver |
| /diagnostics    | diagnostic_msgs/msg/DiagnosticArray | Monitor        |

### 17.3 TF topics

| Topic      | Type                   | Use                |
|------------|------------------------|--------------------|
| /tf        | tf2_msgs/msg/TFMessage | Dynamic transforms |
| /tf_static | tf2_msgs/msg/TFMessage | Static transforms  |

The stationary publisher uses `/tf_static`. A mobile estimator normally uses `/tf` for time-varying pose and may also publish static extrinsics.

### 17.4 Mapping topics

| Topic                        | Type          | Durability when latched |
|------------------------------|---------------|-------------------------|
| /occupied_cells_vis_array    | MarkerArray   | Transient Local         |
| /free_cells_vis_array        | MarkerArray   | Transient Local         |
| /octomap_binary              | Octomap       | Transient Local         |
| /octomap_full                | Octomap       | Transient Local         |
| /octomap_point_cloud_centers | PointCloud2   | Transient Local         |
| /projected_map               | OccupancyGrid | Transient Local         |

Free-space marker publication is disabled by the current configuration.

### 17.5 Services

| Service                    | Type                        | Function     |
|----------------------------|-----------------------------|--------------|
| /octomap_binary            | octomap_msgs/srv/GetOctomap | Compact map  |
| /octomap_full              | octomap_msgs/srv/GetOctomap | Full map     |
| /octomap_server/clear_bbox | Bounding-box query          | Clear region |
| /octomap_server/reset      | std_srvs/srv/Empty          | Reset tree   |

## 17.6 Frames

| Frame          | Authority                       | Meaning                 |
|----------------|---------------------------------|-------------------------|
| unilidar_lidar | Driver header                   | LiDAR cloud coordinates |
| unilidar_imu   | Driver header                   | IMU message coordinates |
| map            | Project or external pose system | OctoMap world frame     |

No project transform connects `unilidar_imu` to `unilidar_lidar`.

## 18. Command and verdict reference

### 18.1 Build and software checks

| Command                                          | Success evidence              |
|--------------------------------------------------|-------------------------------|
| <code>./scripts/docker-build.sh</code>           | Image build completes         |
| <code>./scripts/fetch-dependencies.sh</code>     | Both dependency commits ready |
| <code>./scripts/workspace-build.sh</code>        | Six packages build            |
| <code>./scripts/smoke-test.sh</code>             | SMOKE_TEST_PASS               |
| <code>./scripts/monitor-synthetic-test.sh</code> | MONITOR_SYNTHETIC_HEALTH_PASS |
| <code>./scripts/gui-smoke-test.sh</code>         | GUI_SMOKE_TEST_PASS           |

### 18.2 Runtime checks

| Command                                | Success evidence            |
|----------------------------------------|-----------------------------|
| <code>assert-ros-container.sh</code>   | DOCKER_ROS_RUNTIME_PASS     |
| <code>lidar-launch.sh preflight</code> | LIDAR_CONTAINER_ACCESS_PASS |
| <code>lidar-validate.sh</code>         | LIDAR_DATA_VALIDATION_PASS  |
| <code>verify-docker-only.sh</code>     | DOCKER_ONLY_PIPELINE_PASS   |
| <code>octomap-launch.sh --check</code> | OCTOMAP_LAUNCH_CHECK_PASS   |

### 18.3 Map checks

| Command                              | Success evidence            |
|--------------------------------------|-----------------------------|
| <code>evaluate-octomap.sh</code>     | OCTOMAP_MAPPING_HEALTH_PASS |
| <code>save-octomap.sh</code>         | OCTOMAP_SAVE_PASS           |
| <code>inspect-octomap.sh</code>      | OCTOMAP_INSPECT_PASS        |
| <code>view-octomap.sh --check</code> | VIEW_OCTOMAP_CHECK_PASS     |

### 18.4 Failure verdicts

`DOCKER_RUNTIME_ASSERT_FAIL` means the requested ROS or GUI process is outside the supported container boundary. It is an intentional refusal, not a signal to install ROS on the host.

An absent success verdict must be investigated even if a preceding command printed useful output.

## 19. Versions, compatibility, licensing, and sources

### 19.1 Reproducible version lock

| Component            | Version or identifier                    |
|----------------------|------------------------------------------|
| Base image           | <code>ros:humble-ros-base-jammy</code>   |
| Container OS         | Ubuntu 22.04 Jammy                       |
| ROS 2                | Humble                                   |
| Unitree UniLiDAR SDK | v1.0.16                                  |
| Unitree commit       | 1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6 |
| OctoMap mapping      | 2.3.1                                    |
| OctoMap commit       | f79da9a9a1fcdf82e72dab4df288d6cc27c6e163 |

Validated compatibility includes:

- GCC and G++ 11.4.
- Unitree x86\_64 archive built with GCC 9.4.
- PCL 1.12.
- Eigen 3.4.
- Boost 1.74.
- OctoMap C++ 1.9.7.

- octomap\_msgs 2.0.1.
- octomap\_ros 0.4.4.
- Python 3.10.
- CMake 3.22.
- RViz2 11.2.

These observations describe the validated environment. They do not promise that arbitrary future package combinations remain compatible.

## 19.2 Licensing

Project-authored source and documentation are distributed under the root MIT license.

Fetch upstream source retains its upstream copyright and licensing. The Unitree repository contains its own BSD 3-Clause license, while the upstream ROS 2 driver package metadata declares GPLv3. OctoMap mapping package metadata declares BSD.

Preserve all upstream notices and evaluate distribution obligations from the actual set of source and binaries being distributed. This manual is not legal advice.

## 19.3 Primary technical sources

The maintained source register is `docs/sources.md`. Primary references include:

- ROS 2 Humble release and supported platforms.
- Official ROS Docker images.
- Docker Engine and Compose service security.
- Unitree UniLiDAR SDK v1.0.16.
- Unitree ROS 2 driver documentation.
- Official Unitree 4D LiDAR L1 manual.
- Unitree L1 download center.
- ROS 2 QoS concepts.
- ROS diagnostic message definitions.
- Official Unitree Point-LIO.
- Candidate community Point-LIO ROS 2 port.
- OctoMap mapping 2.3.1.
- Colcon configuration and discovery.

The official L1 manual is the authority for electrical, cabling, interface, and nominal sensor specifications. Project measurements are operational evidence, not replacements for manufacturer specifications.

## 19.4 Source-of-truth paths

| Subject               | Repository path                                     |
|-----------------------|-----------------------------------------------------|
| Dependency commits    | <code>config/dependencies.repos</code>              |
| Container packages    | <code>docker/Dockerfile</code>                      |
| Base service          | <code>docker/compose.yaml</code>                    |
| GUI boundary          | <code>docker/compose.gui.yaml</code>                |
| LiDAR boundary        | <code>docker/compose.lidar.yaml</code>              |
| Colcon paths          | <code>ros2_ws/colcon_defaults.yaml</code>           |
| Driver configuration  | <code>l1_bringup/config/unitree_l1.yaml</code>      |
| Monitor behavior      | <code>l1_monitor/l1_monitor/monitor_node.py</code>  |
| OctoMap configuration | <code>l1_octomap_bringup/config/octomap.yaml</code> |
| Hardware procedure    | <code>docs/hardware-runbook.md</code>               |
| Acceptance record     | <code>docs/validation-matrix.md</code>              |

The package paths in the final four table rows are relative to `ros2_ws/src/`.

## 20. Final engineering principles

The project is organized around several durable principles:

- Keep the Ubuntu host free of the ROS 2 Humble runtime.
- Make every hardware and GUI boundary explicit.
- Prefer evidence over broad permissions or speculative service changes.
- Keep fetched upstream source immutable and adaptations project-owned.
- Keep generated output outside the source tree and outside Git.
- Treat a live node as a process state, not a data-validity result.
- Validate messages, rates, timestamps, frames, fields, and diagnostics.
- Treat TF as a required mapping input, not an OctoMap output.
- Use the stationary identity transform only for a truly stationary sensor.
- Save maps through controlled paths without overwrite.
- Distinguish non-empty map health from geometric accuracy.
- Change one owned layer at a time and update validation with the change.

Following these principles keeps the system reproducible, inspectable, and honest about its current capabilities. That is the foundation required before adding mobile pose estimation or deploying the L1 beyond a controlled bench.